Financial Deposit and Valuation Formulas

Project Documentation

October 10, 2026

1 Recurring Deposit Formula

The value of a recurring deposit (derived in `recurring-deposit-proof.tex`) is calculated as follows:

$$\text{Value} = \frac{Y_d}{T_d} \cdot I \cdot \frac{\left[1 - \left(1 + \frac{C_d \cdot r}{100 \cdot Y_d}\right)^{\frac{T_d}{C_d}}\right] \left(1 + \frac{C_d \cdot r}{100 \cdot Y_d}\right)^{\frac{R_d}{C_d}}}{1 - \left(1 + \frac{C_d \cdot r}{100 \cdot Y_d}\right)^{\frac{R_d}{C_d}}}$$

Parameter Definitions

- I: Installment amount deposited at the end of every payment period R_d
- r: Annual rate of interest (%)
- Y_d: Days in a year
- R_d: Days in a recurring payment period
- C_d: Compounding frequency in days
- T_d: Total tenure in days

2 Fixed Deposit Formula

The maturity amount A for a fixed deposit is given by:

$$A = p \left(1 + \frac{r \cdot C_d}{100 \cdot Y_d}\right)^{\frac{T_d}{C_d}}$$

Parameter Definitions

- A: Final maturity amount
- p: Principal amount deposited
- r: Annual rate of interest (%)
- C_d: Compounding frequency in days
- Y_d: Days in a year
- T_d: Total tenure in days

3 Value at the End of n Years

The discounted present value or value at the end of n years is given by:

$$A(n) = \frac{A}{1 + \frac{n \cdot R_i}{100}}$$

Parameter Definitions

- $A(n)$: Value at the end of n years / time T
- A: Base amount
- n: Number of years
- R_i: Annual discount or interest rate (%)

Markov Decision Process / Planning Problem

1. $s \in \mathbf{S}$ and $a \in \mathbf{A}$
2. Initial state $s_{\text{initial}} \in \mathbf{S}$
3. GoalState $s_{\text{Goal}} \in \mathbf{S}$
4. $\forall (s, a) \in \mathbf{S} \times \mathbf{A}, \exists r \in \mathbb{R}$ and $\exists s' \in \mathbf{S}$:
   $s' = \mathbf{nextState}(s, a)$ (Find out from the environment)
   $r = \mathbf{reward}(s, a)$ (Get from the environment)
5. Planning problem: Find $\mathbf{a}^* = (a_0, a_1, \ldots, a_T)$ and $\mathbf{s}^* = (s_{\text{initial}}, s_1, \ldots, s_T, s_{\text{Goal}})$ to maximize total reward
6. Problem definition: Find the $\mathbf{S}, \mathbf{A}, \mathbf{nextState}$, and $\mathbf{reward}$ functions.
7. **AI**, Please solve this problem.

RAG Evaluation Metrics and System Architecture

System Design & Formulations

October 10, 2026

1. System Architecture Diagram

Below is the schematic representation of the Retrieval-Augmented Generation (RAG) pipeline showing the query Q, document database D, retrieved contexts (C_1, C_2, C_3), and generated answer A.

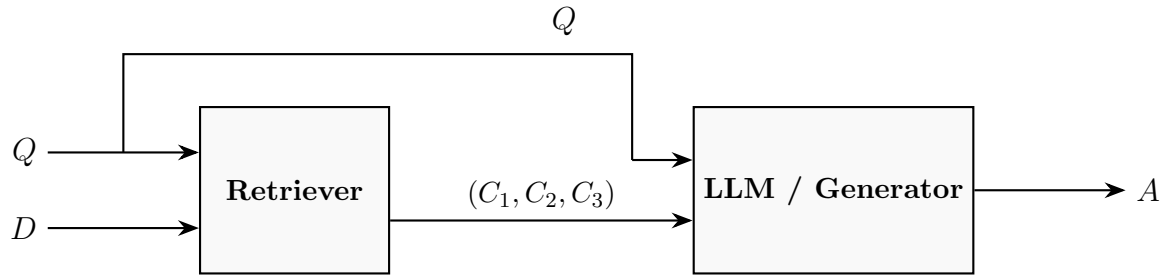


Figure 1: Retrieval-Augmented Generation (RAG) Data Flow Architecture

2. Mathematical Formulations of RAG Metrics

2.1. Faithfulness

$$\text{Faithfulness}(A, C_{1:n}) = \frac{\text{Number of claims in } A \text{ that can be inferred from the context } C_{1:n}}{\text{Total number of claims in } A}$$

2.2. Context Relevance

$$\text{ContextRelevance}(Q, C_{1:n}) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{Q \cdot C_i + 1}{2}\right) = \frac{1}{2}\left(\frac{1}{n}\left(Q \cdot \sum_{i=1}^{n} C_i\right) + 1\right)$$

$$\text{ContextRelevance}(Q, C_{1:n}, \beta) = \frac{\sum_{i=1}^{n} \mathbb{I}(Q \cdot C_i > \beta)}{n} \quad \text{where } \beta \text{ is the bound on the semantic similarity}$$

2.3. Answer Relevance

$$\text{AnswerRelevance}(Q, A) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{Q \cdot Q_i(A) + 1}{2}\right) = \frac{1}{2}\left(\frac{1}{n}\left(Q \cdot \sum_{i=1}^{n} Q_i(A)\right) + 1\right)$$

2.4. Context Recall

$$\text{ContextRecall}(GT, C_{1:n}) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{GT \cdot C_i + 1}{2}\right) = \frac{1}{2}\left(\frac{1}{n}\left(GT \cdot \sum_{i=1}^{n} C_i\right) + 1\right)$$

Best Goal Search Algorithm

Typeset from Manuscript Notes

October 10, 2026

1 Overview & Time Complexity

`GoalSearch.BestSearch` navigates state transitions from an initial state S_0 towards a goal state S_g. It is an iterative depth-first search over an explicit stack. If S_g is reached, that path is returned. Otherwise, it returns the path to the leaf state with the highest balance among the leaves it visits.

$$\underline{\text{Best Goal Search}} \implies \text{Complexity } O\big(|A|^{\text{MAX-DEPTH}}\big)$$

Where:

- $A = (A_0, \ldots, A_{|A|-1})$: The ordered list of available actions (deposit schemes), indexed by a.
- $|A|$: The branching factor (number of actions).
- MAX-DEPTH: The deepest stack reached before the reachability test fails (bounded by the goal time horizon).

2 Interface

- **Transition**: $(S', c) = T(S, A_a)$ returns the successor state S' and the number of times c that action A_a is repeated to reach it.
- **Reachability test**: $isGR(S', S_g)$ returns one of:
  - `GOAL_REACHED`: S' satisfies the goal.
  - `GOAL_NOT_YET_REACHED`: S' can be extended further.
  - `GOAL_NOT_REACHABLE`: S' overshoots the goal and is discarded.
- **Stack entries**: each entry is $((a, c), S')$: the action index, its repeat count, and the state it produced.

3 Typeset Pseudocode

```
GoalSearch(Sg, A, T, isGR)

BestSearch(S0):
    S = S0, stack = [], a = 0
    best = [], bestBalance = 0
    prev = GOAL_NOT_YET_REACHED

    loop:
        (S', c) = T(S, A[a])
        st = isGR(S', Sg)

        if st == GOAL_NOT_YET_REACHED:       # go one level deeper
            stack.push(((a, c), S'))
            S = S'
            a = 0

        elif st == GOAL_NOT_REACHABLE:       # try the next sibling action
            a = a + 1
            if a >= |A|:                     # all siblings tried: backtrack
                if stack is empty:
                    break
                ((a, _), _) = stack.pop()
                a = a + 1
                if stack is empty:
                    S = S0
                    break
                S = top(stack).state
            elif prev == GOAL_NOT_YET_REACHED:
                # S is a leaf: keep the path if it has the best balance
                if S.balance > bestBalance:
                    bestBalance = S.balance
                    best = copy(stack)

        else:
            stack.push(((a, c), S'))
            best = copy(stack)
            break

        prev = st

    L = copy(best)
    return st

GetFinalList():
    if L is empty:
        return (L, Sg.balance - S.balance)
    return (L, Sg.balance - L[-1].state.balance)

# L has the answer; the second value is the shortfall to the goal
```

Listing 1: Best Goal Search Pseudocode

4 Step-by-Step Logic

1. **Transition**: Apply action A_a to the current state S to get $(S', c) = T(S, A_a)$ and classify S' with $isGR(S', S_g)$.
2. **Descend**: If S' is `GOAL_NOT_YET_REACHED`, push $((a, c), S')$, make S' the current state and restart from the first action $a = 0$.
3. **Leaf Evaluation**: If S' is `GOAL_NOT_REACHABLE` right after a descent, the current state S is a leaf. If its balance beats the best seen so far, record a copy of the stack as the best path.

4. **Next Sibling**: After a `GOAL_NOT_REACHABLE` result, move to the next action $a+1$ at the same depth.
5. **Backtracking**: When every action at a depth has been tried, pop the top entry and continue from the action after the popped one, with the new stack top as the current state. The search stops once the stack becomes empty.
6. **Goal Reached**: If S' is `GOAL_REACHED`, push $((a, c), S')$, take the whole stack as the answer and stop.
7. **Result**: The stored path L is returned by `GetFinalList` together with the shortfall S_g.balance $-$ S_{end}.balance, where S_{end} is the last state on L, or the initial state when L is empty.

Derivation of Recurring Deposit Yield Formula

Mathematical Typesetting of Handwritten Notes (IMG_20261004_090254)

Notation & Variables

I (or A)	:	Installment amount deposited at the end of every payment period R_d.
R_d	:	Duration of each recurring deposit installment period (in days).
C_d	:	Duration of each compounding period (in days).
T_d	:	Total tenure / duration of the recurring deposit (in days).
r	:	Annual rate of interest (%).
Y_d	:	Total duration of one year (in days, e.g., 365).

Step-by-Step Mathematical Derivation

Below is the complete, line-by-line LaTeX transcription and typesetting of the mathematical derivation from IMG_20261004_090254.

1. Initial Compound Interest Summation

The total accrued value across all K installment periods is formulated as the sum of individually compounded installments, adjusted by the annualized day ratio $\frac{Y_d}{T_d}$:

$$\left[\sum_{K=0}^{\left(\frac{T_d}{R_d}-1\right)} I\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{T_d-KR_d}{C_d}}\right]\times\frac{Y_d}{T_d} \tag{1}$$

2. Factoring Constant Terms

Decomposing the exponent $\frac{T_d-KR_d}{C_d}=\frac{T_d}{C_d}-\frac{KR_d}{C_d}$ allows splitting the compound term into product factors:

$$=\frac{Y_d}{T_d}\times I\sum_{K=0}^{\left(\frac{T_d}{R_d}-1\right)}\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{T_d}{C_d}}\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{-\frac{KR_d}{C_d}} \tag{2}$$

Factoring out the constant factor $\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{T_d}{C_d}}$ outside the summation yields:

$$=\frac{Y_d}{T_d}\times I\times\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{T_d}{C_d}}\sum_{K=0}^{\left(\frac{T_d}{R_d}-1\right)}\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{-\frac{KR_d}{C_d}} \tag{3}$$

3. Evaluating the Finite Geometric Series

The remaining summation represents a finite geometric series $\sum_{K=0}^{N-1} x^K = \frac{x^N-1}{x-1}$, where $x = \left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{-\frac{R_d}{C_d}}$ and $N = \frac{T_d}{R_d}$:

$$= \frac{Y_d}{T_d} \times I \times \left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{T_d}{C_d}} \left[\frac{\left[\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{-\frac{R_d}{C_d}}\right]^{\frac{T_d}{R_d}} - 1}{\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{-\frac{R_d}{C_d}} - 1} \right] \tag{4}$$

Simplifying the exponent in the numerator $\left(-\frac{R_d}{C_d}\right) \times \left(\frac{T_d}{R_d}\right) = -\frac{T_d}{C_d}$:

$$= \frac{Y_d}{T_d} \times I \times \left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{T_d}{C_d}} \left[\frac{\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{-\frac{T_d}{C_d}} - 1}{\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{-\frac{R_d}{C_d}} - 1} \right] \tag{5}$$

4. Algebraic Simplification to Positive Exponent Form

Multiplying the numerator and denominator by positive exponential scaling factors eliminates negative exponents:

$$= \frac{Y_d}{T_d} \times I \times \left[\frac{\left[1 - \left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{T_d}{C_d}}\right]\left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{R_d}{C_d}}}{\left[1 - \left(1+\frac{C_d}{Y_d}\frac{r}{100}\right)^{\frac{R_d}{C_d}}\right]} \right] \tag{6}$$

Final Typeset Closed-Form Result

$$\text{Value} = \frac{Y_d}{T_d} \cdot I \cdot \frac{\left[1 - \left(1+\frac{C_d \cdot r}{100 \cdot Y_d}\right)^{\frac{T_d}{C_d}}\right]\left(1+\frac{C_d \cdot r}{100 \cdot Y_d}\right)^{\frac{R_d}{C_d}}}{1 - \left(1+\frac{C_d \cdot r}{100 \cdot Y_d}\right)^{\frac{R_d}{C_d}}}$$

LangGraph Agent Graphs

Project Documentation

October 10, 2026

1 Overview

The three LangGraph applications in this project are built by `buildAdminAgentGraph()` in their respective files. All three compile the same `StateGraph(AdminAgentState)` topology. Only the tools bound to the LLM and the model differ:

File	LLM model	Tools
`DatabaseCreator.py`	`gpt-4.1-mini`	6
`FinancialGeneralInformationChatBot.py`	`gpt-4.1`	2
`OptimalBankDepositFinderAgent.py`	`gpt-5`	7

Shared structure

- **State**: `AdminAgentState` holds a single `messages` list, merged with `operator.add`.
- **Nodes**:
  - `AdminAgent` (`AdminAgentNode`): adds the system prompt once, then calls the tool-bound LLM.
  - `tool` (`ToolsNode`): runs every tool call in the last AI message and returns the `ToolMessage`s.
  - `ExitGuardrail` (`ExitGuardRailNode`): checks the final answer against the last human message, and replaces it with a refusal if it fails.
- **Conditional edges** (dashed):
  - From `__start__`, `inputGuardRailCondition` returns `continue` (input passes the guardrail) or `end`.
  - From `AdminAgent`, `where_to_go` returns `continue` (the AI message has `tool_calls`) or `end`.
- **Fixed edges** (solid): `tool` → `AdminAgent` and `ExitGuardrail` → `__end__`.

2 DatabaseCreator.py

Admin agent that fills the SQLite database from the RAG stores (model `gpt-4.1-mini`).

```
flowchart TD
    __start__([__start__])
    AdminAgent[AdminAgent]
    tool["tool<br/>UpdateSqlLiteTableStructure<br/>GetSqlLiteTablesStructure<br/>
    UpdateSqlLiteTableData<br/>GetSqlLiteTableData<br/>
    GetRequiredBankDepositDataFromTheRagTool<br/>GetRequiredMutualFundDataFromTheRagTool"]
    ExitGuardrail[ExitGuardrail]
    __end__([__end__])

    __start__ -. continue .-> AdminAgent
    __start__ -. end .-> __end__
    AdminAgent -. continue .-> tool
    AdminAgent -. end .-> ExitGuardrail
    tool --> AdminAgent
    ExitGuardrail --> __end__
```

Listing 2: Mermaid: DatabaseCreator.py

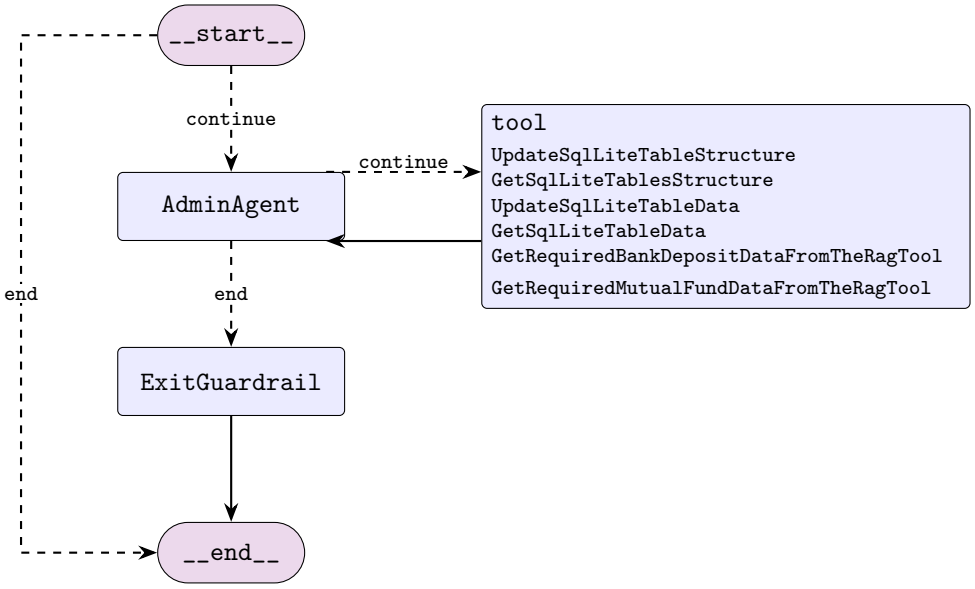


Figure 2: Graph of `DatabaseCreator.py`

3 FinancialGeneralInformationChatBot.py

Chatbot that answers general bank deposit and mutual fund questions from the RAG stores (model gpt-4.1).

```
flowchart TD
    __start__([__start__])
    AdminAgent[AdminAgent]
    tool["tool<br/>GetRequiredBankDepositDataFromTheRagTool<br/>
    GetRequiredMutualFundDataFromTheRagTool"]
    ExitGuardrail[ExitGuardrail]
    __end__([__end__])

    __start__ -. continue .-> AdminAgent
    __start__ -. end .-> __end__
    AdminAgent -. continue .-> tool
    AdminAgent -. end .-> ExitGuardrail
    tool --> AdminAgent
    ExitGuardrail --> __end__
```

Listing 3: Mermaid: FinancialGeneralInformationChatBot.py

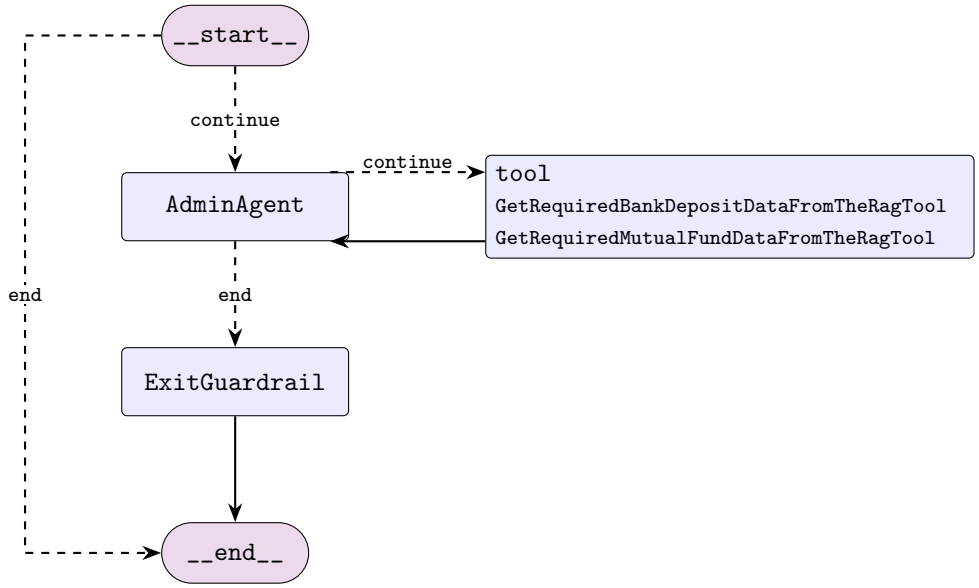


Figure 3: Graph of FinancialGeneralInformationChatBot.py

4 OptimalBankDepositFinderAgent.py

Agent that reads the deposit schemes from SQLite and finds the optimal fixed and recurring deposit sequence for a goal (model `gpt-5`). The tool `WriteMermaidDiagramPage` renders mermaid descriptions of the result as diagrams in an HTML page in the `output` folder.

```
flowchart TD
    __start__([__start__])
    AdminAgent[AdminAgent]
    tool["tool<br/>GetSqlLiteTablesStructure<br/>GetSqlLiteTableData<br/>
    OptimalActionFixedDepositTool<br/>OptimalActionRecurringDepositTool<br/>
    get_india_inflation<br/>GetFinalValueConsideringInflation<br/>WriteMermaidDiagramPage"]
    ExitGuardrail[ExitGuardrail]
    __end__([__end__])

    __start__ -. continue .-> AdminAgent
    __start__ -. end .-> __end__
    AdminAgent -. continue .-> tool
    AdminAgent -. end .-> ExitGuardrail
    tool --> AdminAgent
    ExitGuardrail --> __end__
```

Listing 4: Mermaid: OptimalBankDepositFinderAgent.py

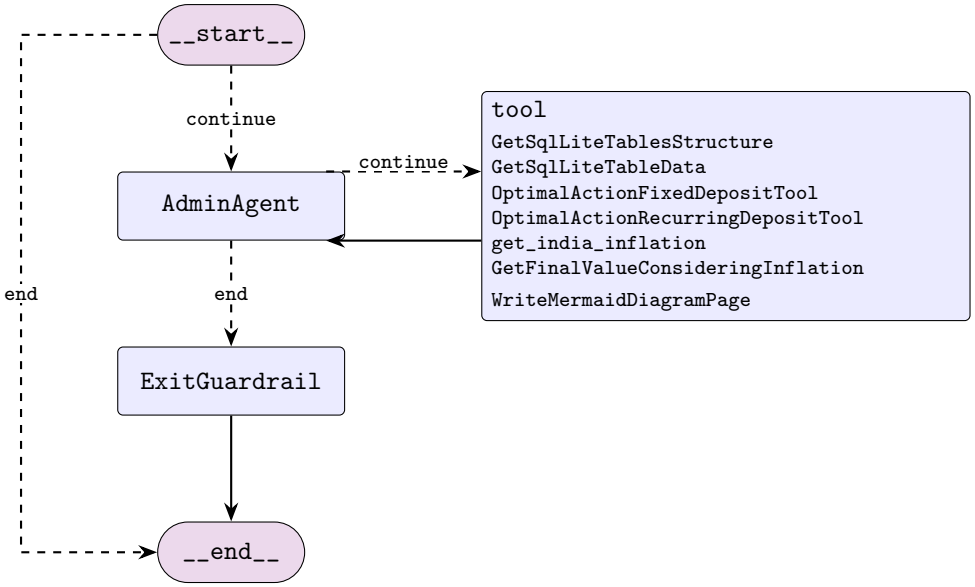


Figure 4: Graph of `OptimalBankDepositFinderAgent.py`